Image classification

Jamshid Dehmeshki

Task 1 : Build a CNN Classifier for Synthetic Shapes

- To create and train a Convolutional Neural Network (CNN) using TensorFlow and OpenCV to classify synthetic images of **circles** and **squares** — with added noise and rotation for realism.

Task Description

- In this assignment, you will build a Convolutional Neural Network (CNN) using TensorFlow (Keras) to classify simple geometric shapes — circles and squares — which are generated synthetically using OpenCV.
- You'll train the model on randomly generated shapes with optional noise and rotation, evaluate its accuracy, and visualize both training samples and final predictions.

Learning Objectives

By completing this task, you will:

- **Deepen your understanding of:**
  - Convolutional Neural Networks (CNNs)
  - Classification problems with image data
  - Synthetic data generation using OpenCV
- **Practice essential ML skills:**
  - Building and training models using **Keras/TensorFlow**
  - Data preprocessing and augmentation
  - Evaluating model performance (loss, accuracy, and confidence scores)
  - Using visualisation for debugging and validation
- **Learn how to:**
  - Generate and label images with OpenCV
  - Add random noise and rotation to simulate real-world variation
  - Control training conditions using feature toggles
  - Display classification predictions with confidence levels

Deliverables

A working Python script (BasicCNN.py) that:

- Creates synthetic shape data
- Trains a CNN
- Evaluates accuracy and confidence
- Displays visual results using OpenCV

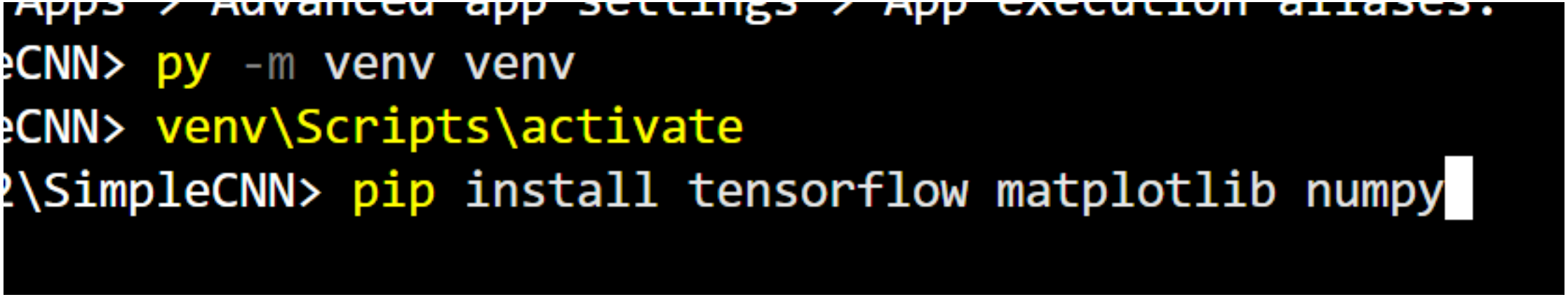
Architecture Overview:

Input:	28x28x1 grayscale image
↓ Conv2D	16 filters, 3x3, ReLU
↓ MaxPooling2D (2x2)	
↓ Conv2D	32 filters, 3x3, ReLU
↓ MaxPooling2D (2x2)	
↓ Flatten	
↓ Dense	64 units, ReLU
↓ Dense	2 units (Circle/Square), Softmax

Create virtual environment in visual code

- **Download BasicCNN.py from canvas**
- Create a folder (e.g. SimpleCNN)
- Go to view/terminal and type the below pip …… (make sure you are in powershell (see right side of terminal before +):

- **Also, pip install scikit-learn**





Components of code

BasicCNN.py

1. Imports and Constants

- We use **OpenCV** to generate and draw shapes
- **TensorFlow/Keras** is used for building and training the CNN
- **NumPy** helps with image arrays
- **scikit-learn** helps split the dataset into training and testing

2. Global Settings

- Defines image size and number of samples
- You can toggle rotation/noise ON/OFF for experiments

3. Image Generators

generate_circle() and generate_square()

- Each generates a grayscale 28×28 image
- Adds:
  - Random **size**
  - Random **position**
  - Optional **rotation** and **noise**
- Normalises pixels to range [0, 1] for CNN input
- apply_random_rotation()
- apply_random_noise()

4. Dataset Creation

create_dataset()

- Calls the generators to make:
  - 500 circle images labeled 0
  - 500 square images labeled 1
- Converts to proper shape: (num_samples, 28, 28, 1)
- Labels are one-hot encoded using to_categorical() (e.g [0,1] and [1,0])
- Data is split: **80% training**, **20% testing**

5. CNN Model Structure

- Conv2D → MaxPool → Conv2D → MaxPool → Flatten → Dense → Dense
- Learn local pattern (edges, shapes)
- Final Dense layer outputs 2 class probabilities via softmax

6. Training the Model

- model.fit(X_train, y_train, epochs=5)
  - Trains for 5 epochs
  - Uses test data for live validation during training

7. Saving and Loading Model

- model.save(...)
  - Saves to shape_classifier.keras
- model = load_model(...)
  - Reloads to test model's persistence and reusability

8. Evaluation

- loss, accuracy = model.evaluate(X_test, y_test)
  - Reports total loss and accuracy on the entire test set

9. Testing with New Images

- test_images = [generate_circle(), generate_square()]
  - Generates two new random shapes (never seen during training)
  - Runs prediction:
  - Prints **true label**, **predicted label**
  - Displays **confidence score**
  - Shows ✅ or ❌ depending on correctness
  - Uses cv2.imshow() to display result visually

10. Extra: Show Training/Test Samples

- Shows a few **training and test images** before training
- Enlarged for clarity using cv2.resize



Summery

Generate Data → Train CNN → Evaluate Model → Predict on New Data → Visualise

Final cocd: BasicCNN.py

includes:

- Toggles for noise and rotation (APPLY_ROTATION, APPLY_NOISE)
- Random size and position for circles and squares
- Training and test image preview using cv2.imshow()
- Confidence score printed and displayed with each prediction
- CNN model with two Conv2D + MaxPool2D layers and softmax output
- Dataset split using train_test_split from scikit-learn
- Final predictions compared against true labels with ✅/❌

Exercise 1: Experiment with Noise and Rotation

- Toggle the following settings in the code and record how the accuracy and predictions change:
- APPLY_ROTATION = True / False
- APPLY_NOISE = True / False

Questions:

- What happens to accuracy when **only noise** is applied?
- What happens when **only rotation** is applied?
- Which shape is harder to classify under each condition?

Exercise 2: Improve Accuracy Under Noise/Rotation

- The current model might struggle when noise and rotation are both applied. Try the following to improve accuracy:
- Increase training size
- Add more training epochs
- Make the CNN deeper
  - Add another Conv2D + MaxPooling2D block
  - Increase number of filters (e.g. 32 → 64)
  - Try dropout or batch normalization (advanced)

Exercise 3: Add More Shape Classes (Advanced)

- Add a new shape: **triangle**
- Label it as class 2
- Update:
  - Image generator
  - Labels
  - CNN output layer to Dense(3, activation='softmax')
- **Challenge:**
- Can your CNN classify **3 shapes** with high accuracy?